System Design, Architecture Trade-offs,

and Technical Decision-Making

Cross-Project Engineering Case Study

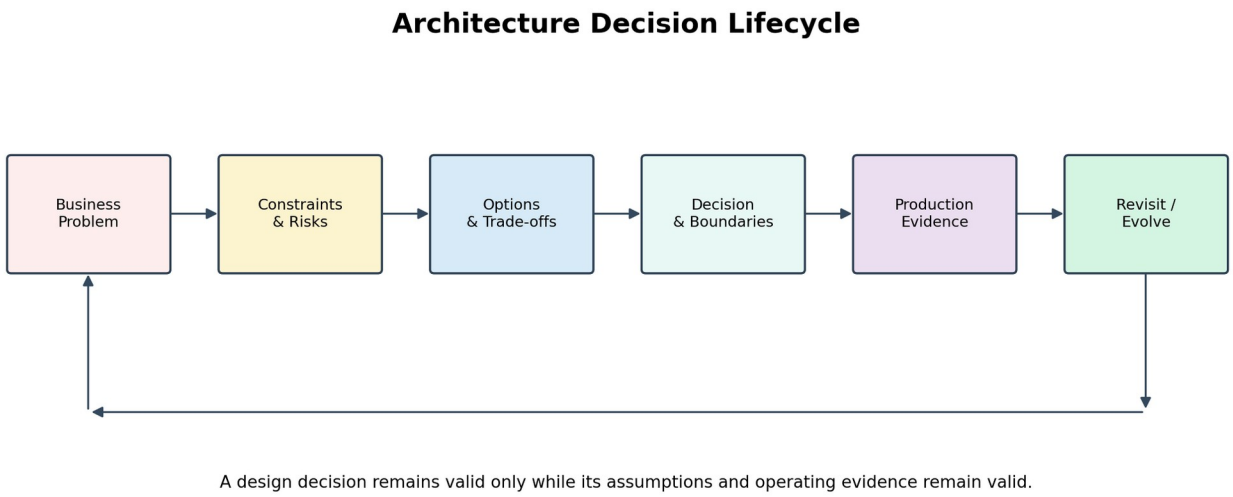


Figure 1. Architecture decisions are continuously validated against production evidence.

Author	Venkata Lakshmi Parimala Pasupuleti
Domains	Affordable Housing, data engineering, backend services, asynchronous systems, cloud storage, and production AI
Primary Purpose	Document how business constraints, operational risk, and evidence drive technical design choices
Experience Basis	Income Limits, RabbitMQ, document migration, database rollout, AWS S3 integration, LeaseI, AgentProbe, SQL, release engineering, and production operations
Decision Principle	Use the simplest design that satisfies correctness, scale, reliability, and ownership requirements
Scope Discipline	Direct experience, generalized engineering patterns, and proposed modernization are clearly distinguished

Document Map

1. Executive Summary
2. Experience Basis and Scope
3. Engineering Decision Philosophy
4. Business Context Before Technology
5. Requirements, Constraints, and Risk
6. Architecture Discovery and System Boundaries
7. System-Level Architecture
8. Data Architecture and Effective Dating
9. Staging, Transformation, and Publication
10. Stored Procedures versus Application Logic
11. Synchronous versus Asynchronous Processing
12. RabbitMQ and Worker Design
13. API and Service Integration
14. Configuration and Cross-Module Dependencies
15. Document Platform Contracts
16. Cloud Object Storage Integration
17. RAG versus Structured Tool Calls
18. LangGraph and Agent Orchestration
19. Deterministic versus Probabilistic Responsibility
20. Reliability and Failure Handling
21. Performance and Scalability
22. Security, Privacy, and Auditability
23. Observability and Production Evidence
24. Deployment, Migration, and Rollback
25. Architecture Evolution and Modernization
26. Decision Records and Reusable Evaluation Framework
27. Common Architecture Anti-Patterns
28. Lessons Learned and Technical Decision Checklist

SCOPE	This is a cross-project architecture synthesis. It does not assert that all systems were deployed as one unified service topology, and it avoids inventing proprietary service names or exact infrastructure details.

POSITION	Senior technical decision-making is the discipline of connecting business impact, system behavior, trade-offs, and production evidence - not merely selecting technologies.

1. Executive Summary

The central theme across these engineering experiences is that architecture is a sequence of decisions made under real constraints. Affordable Housing systems require regulatory correctness, historical accuracy, reliable annual data updates, stable document generation, safe releases, and rapid production diagnosis. AI systems add probabilistic behavior, evaluation needs, and new security boundaries, but they do not remove the need for disciplined engineering.

The most important decisions were rarely about a product name. They were decisions such as whether work belonged in a synchronous request or an asynchronous queue, whether logic should live in SQL or an application service, whether changing information should be embedded or fetched through a live tool, whether a migration should use a phased rollout, and where a human or deterministic engine must remain authoritative.

Primary architecture outcomes

- Start with business behavior, data ownership, failure impact, and operational constraints.
- Define clear boundaries between source data, transformation, deterministic business logic, configuration, and customer outputs.
- Use asynchronous processing when work is long-running, bursty, retryable, or independently scalable.
- Keep authoritative and changing values in structured systems or APIs instead of treating everything as RAG content.
- Use AI for retrieval, interpretation, explanation, and investigation while deterministic systems retain regulated decisions.
- Design validation, observability, deployment, and rollback as part of the architecture, not as later operational additions.
- Revisit decisions when production evidence shows that assumptions have changed.

2. Experience Basis and Scope

System / Experience	Architectural decisions demonstrated	Scope treatment
Income Limits Processing	Staging, transformation, effective dating, stored procedures, reconciliation, downstream publication	Direct long-term production ownership.
Annual Gross Rent with RabbitMQ	Async processing, worker scaling, retry, failure isolation, transaction boundaries	Direct project experience; detailed queue patterns are generalized where historical implementation specifics were not confirmed.
Document Platform Migration	Schema contracts, compatibility, mappings, regeneration risk, cross-team change	Direct migration and production-support experience.
Version 2 Database Rollout	Phased rollout, migration validation, dumps, timestamp analysis, rollback readiness	Direct release and incident-investigation experience.
AWS S3 Document Migration	Shared framework integration, document classification, metadata and validation	Direct Affordable Housing integration ownership, not ownership of the full platform framework.
LeaseI	Four-source model, LangGraph, RAG, tools, validation, escalation, read-only safety	Built and target capabilities remain explicitly separated.
AgentProbe	AI evaluation, RAG tracing, adaptive testing, regression-oriented design	Project implementation plus production-oriented extensions. Memory-safe scope is preserved.
SQL and Production Operations	Data integrity, execution plans, migration safety, evidence-driven debugging	Direct technical strength and repeated production experience.

3. Engineering Decision Philosophy

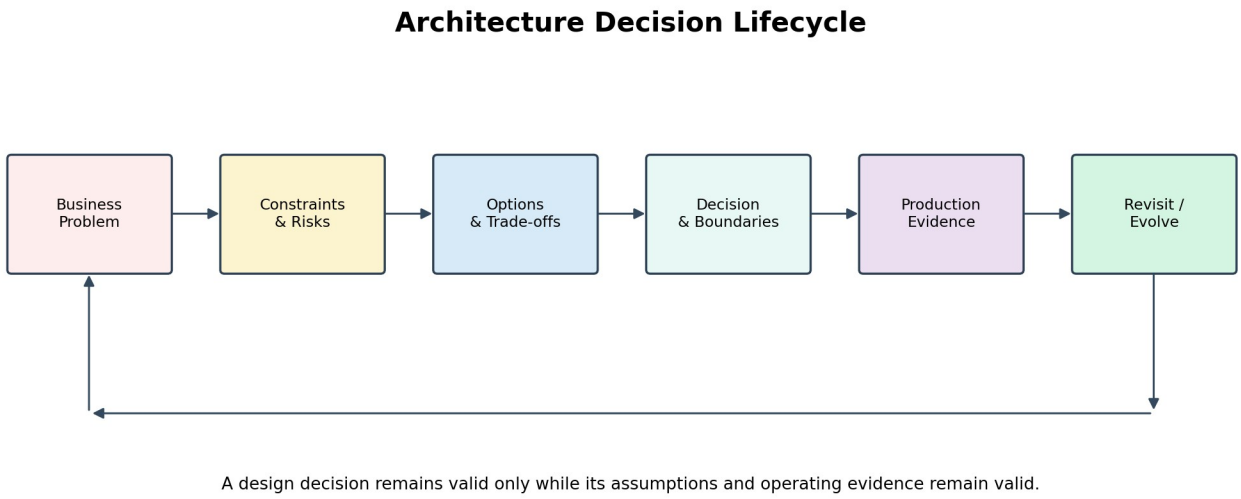


Figure 2. A design is a living decision whose assumptions must be validated in operation.

A technical decision begins with a problem statement and ends only after the system has produced operational evidence. The initial design captures assumptions: expected volume, latency, data quality, failure modes, team skill, and deployment environment. Production then confirms or disproves those assumptions.

Decision dimension	Question	Example
Business value	What outcome must improve?	Reduce annual processing time, prevent incorrect eligibility, or shorten support investigation.
Correctness	What must never be wrong?	Effective dates, jurisdiction mappings, financial calculations, merge-field values, and tenant access boundaries.
Scale	Where will volume or concurrency appear?	Approximately 180,000 transformed records, thousands of gross-rent updates, or large document sets.
Reliability	How does the system recover?	Retry, idempotency, checkpoints, reconciliation, rollback, or human escalation.
Ownership	Who can operate and change it safely?	Module team, shared platform team, database team, support, or AI workflow owner.

Evolution is evaluated separately: stable contracts, adapters, metrics, and phased modernization allow internals to change without a risky rewrite.

4. Business Context Before Technology

Technology decisions in Affordable Housing cannot be evaluated without understanding the domain. Eligibility, income limits, rents, HAP, tax-credit restrictions, documents, and property settings form an interconnected workflow. An apparently local change can affect downstream calculations and customer outputs.

- Annual regulatory data introduces effective dates, jurisdiction mappings, household-size dimensions, and historical-version requirements.
- Eligibility and financial workflows require deterministic, auditable logic.
- Documents are customer-facing records whose correctness depends on stable merge-field contracts and source data.
- Property and global settings can override or alter behavior across module boundaries.
- Support incidents frequently require product knowledge, regulatory knowledge, historical incidents, and live structured data.
- Releases often combine application code, database changes, data movement, and cross-team dependencies.

This context explains why database design, configuration visibility, release validation, and evidence collection were often more important than introducing a new framework.

5. Requirements, Constraints, and Risk

Requirement class	Representative constraints	Architectural implication
Functional	Import current limits, calculate eligibility, generate documents, investigate support tickets	Define explicit workflows, inputs, outputs, and ownership boundaries.
Correctness	No incorrect jurisdiction, household-size, effective-date, or financial result	Layered validation, deterministic rules, audit evidence, and reconciliation.
Performance	Large yearly transformations and thousands of updates must complete in an operational window	Set-based SQL, batching, asynchronous workers, indexes, and capacity measurement.
Availability	Customer workflows should not block on long-running processing	Queue-based execution, resumability, and controlled background work.
Compatibility	Existing templates, records, and APIs must continue to function during migration	Stable contracts, mapping layers, dual-read or phased rollout when justified.
Security	Resident and property data require controlled access and auditability	Least privilege, tenant isolation, read-only diagnostic tools, and logging.
Maintainability	Critical systems should not depend on one engineer	Documentation, shared runbooks, explicit contracts, and knowledge transfer.

RISK MODEL

High-risk changes combine broad blast radius, difficult rollback, weak observability, and cross-team dependencies. Those changes require stronger evidence and staged release controls.

6. Architecture Discovery and System Boundaries

Before proposing a boundary, the existing flow must be reconstructed. In legacy enterprise systems, architecture is frequently encoded across stored procedures, schemas, settings, jobs, document templates, and operational knowledge rather than one current diagram.

1. Identify the business event that starts the workflow and the customer-visible result that ends it.
2. Trace authoritative data sources, transformations, settings, and downstream consumers.
3. Identify ownership boundaries: module team, platform team, database team, or external authority.
4. List failure states and determine which component can detect and recover from each one.
5. Determine whether a boundary represents a true independent capability or only a code organization preference.
6. Capture temporal behavior: effective dates, historical versions, release windows, and migration periods.
7. Validate the reconstructed architecture with logs, queries, data samples, release artifacts, and subject-matter experts.

BOUNDARY	A service boundary should align with independently owned business capability, data responsibility, scaling behavior, or failure isolation. Splitting code without those properties creates a distributed monolith.

7. System-Level Architecture

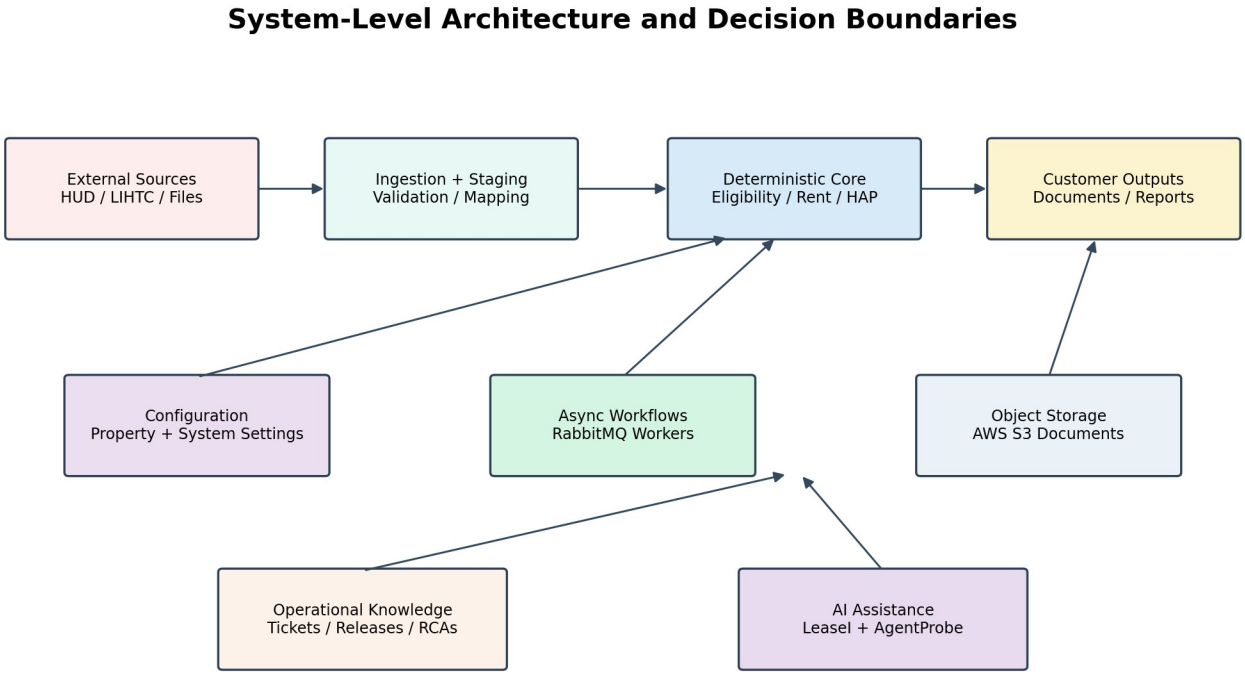


Figure 3. Logical system view connecting regulatory data, deterministic workflows, configuration, documents, operations, and AI assistance.

This diagram is a logical architecture, not a claim that the production platform used these exact deployable service boundaries. It illustrates the decision boundaries that repeatedly appeared across the projects.

Layer	Responsibility	Key design concern
External and Regulatory Sources	HUD/LIHTC data, files, APIs, and governing rules	Authority, version, jurisdiction, effective date, and source traceability.
Ingestion and Staging	Parse, normalize, validate, and prepare source data	Repeatability, rejects, mapping visibility, and reconciliation.
Deterministic Core	Eligibility, rent, HAP, tax-credit, and compliance logic	Correctness, auditability, temporal behavior, and performance.
Configuration	Property settings, system settings, flags, and overrides	Precedence, discoverability, change history, and cross-module impact.
Async Processing	Workers for long-running or high-volume operations	Idempotency, retry, backpressure, and status visibility.
Document and Object Storage	Templates, merge values, generated files, and S3 objects	Contract compatibility, metadata integrity, access, and regeneration.
Operational Knowledge	Tickets, releases, root causes, and runbooks	Trust level, freshness, searchability, and feedback loops.

8. Data Architecture and Effective Dating

Relational data remained the correct foundation for the core platform because the domain depends on structured relationships, deterministic queries, transactions, historical versions, and auditable transformations. Effective dating is a first-class design concern rather than an extra column added after implementation.

Data concept	Design choice	Reason
Source record	Preserve source identity and import batch	Supports lineage, replay, and reconciliation.
Staging record	Normalize without immediately overwriting published data	Separates parsing failures from business transformation failures.
Published rule or limit	Store effective period and dimensional keys	Allows the correct historical value to be selected for a certification date.
Configuration	Store scope, value, source, and change metadata	Supports precedence analysis and cross-team debugging.
Generated document metadata	Reference source entity, template version, and storage object	Supports retrieval, audit, and controlled regeneration.
Investigation evidence	Store source type, retrieval time, and reference	Makes support conclusions reproducible and auditable.

- Prefer immutable historical versions or explicit validity ranges for regulated values.
- Avoid overwriting prior-year data when historical certifications must remain reproducible.
- Use unique constraints to prevent duplicate dimensional records.
- Index by actual access patterns such as jurisdiction, household size, effective date, property, or module.
- Reconcile source, staging, and published counts using known transformation rules instead of assuming one-to-one row counts.

9. Staging, Transformation, and Publication

The Income Limits workflow demonstrated why ingestion, transformation, and publication should be separated. A few thousand source records could expand to approximately 180,000 domain-specific records after applying geography, household-size, program, and percentage dimensions. Directly loading source data into final tables would make partial failures and validation difficult to isolate.

1. Land the source file or API response with batch metadata and checksum.
2. Validate file structure, required columns, data types, and basic completeness.
3. Load staging tables without mutating the current published dataset.
4. Normalize geographic and program identifiers through explicit mappings.
5. Apply set-based transformations and generate required dimensional records.
6. Run count, uniqueness, range, effective-date, and cross-table reconciliation checks.
7. Publish within a controlled transaction or version switch.
8. Execute downstream scenario validation before declaring the import complete.

TRADE-OFF	Staging adds tables and process steps, but it greatly improves diagnosability, repeatability, and rollback safety for business-critical data.

10. Stored Procedures versus Application Logic

Stored procedures were effective for large set-based transformations, data-local operations, and transactional updates. The decision was not that all business logic belongs in the database. It was that certain workloads benefit from executing close to the data with a stable transaction boundary.

Use stored procedures when	Use application logic when	Shared requirement
The operation is set-based and touches many rows.	The workflow coordinates APIs, queues, or multiple external systems.	Inputs, outputs, errors, and ownership must be explicit.
A single database transaction is the natural consistency boundary.	The logic benefits from language libraries, richer domain models, or independent deployment.	Both paths require automated verification and observability.
Data movement would be more expensive than in-database computation.	The logic needs horizontal scaling independent of the database.	Avoid duplicating the same rule in both locations.
Existing operational expertise and deployment process support it.	The team can operate the service boundary and its failure modes.	Measure performance and maintainability rather than following ideology.

A modernized design may keep heavy transformation in SQL while moving orchestration, external integrations, and workflow state into application services. This is an incremental boundary, not a requirement to rewrite proven database logic.

11. Synchronous versus Asynchronous Processing

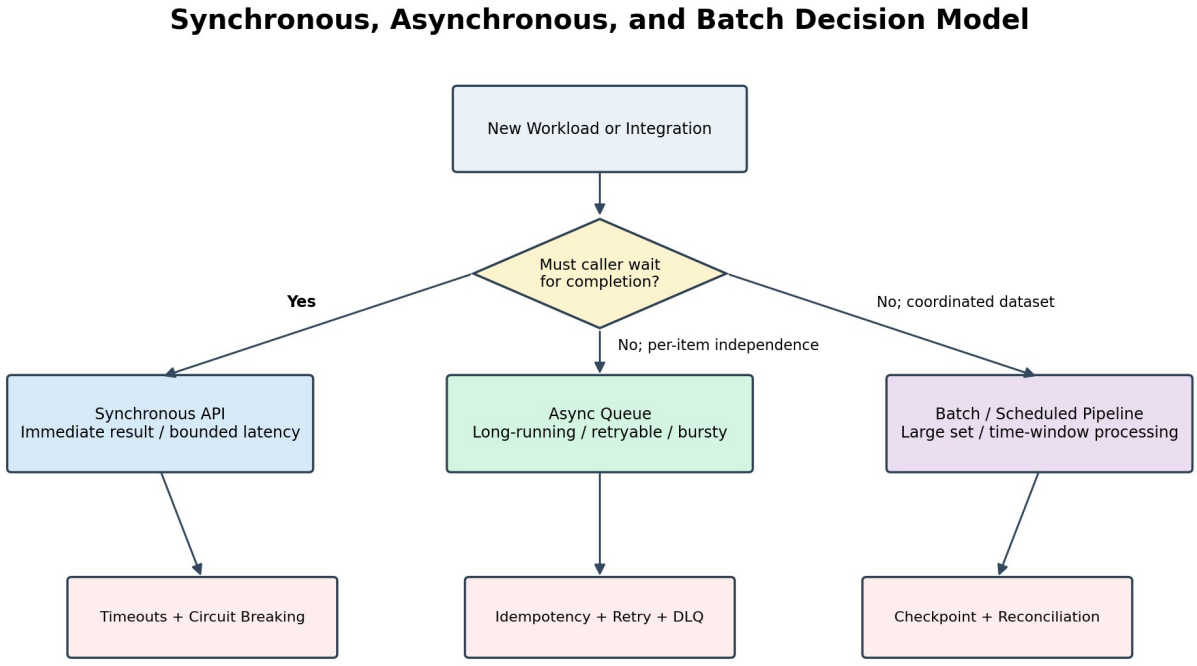


Figure 4. Choose communication style from user expectation, workload behavior, and recovery needs.

Synchronous processing is appropriate when the caller needs an immediate result and the operation has predictable bounded latency. Asynchronous processing is appropriate when work is long-running, bursty, independently retryable, or able to complete after the user request. Batch processing is appropriate when a coordinated dataset must be transformed and reconciled as a unit.

Decision factor	Synchronous	Asynchronous queue	Batch pipeline
User expectation	Immediate response	Accepted / processing status	Scheduled or operator-controlled completion
Failure handling	Timeout, retry, circuit breaker	Retry, idempotency, DLQ, replay	Checkpoint, restart, reconcile, republish
Scaling	Request-serving capacity	Independent worker capacity	Database and job execution capacity
Best example	Read-only lookup or small validation	Thousands of gross-rent updates	Annual income-limit transformation
Primary risk	Cascading latency or timeout	Duplicate delivery and hidden backlog	Partial dataset and difficult rollback

12. RabbitMQ and Worker Design

RabbitMQ was introduced for annual gross-rent processing because thousands of updates should not block an interactive request. The architectural value was decoupling the request or scheduling event from the execution capacity of workers.

- Publish a stable message contract containing identifiers and the minimum required execution context.
- Persist business state before acknowledging completion.
- Assume at-least-once delivery and design consumers to be idempotent.
- Classify failures into transient, permanent, and operator-review categories.
- Use bounded retries and dead-letter handling rather than infinite retry loops.
- Monitor queue depth, oldest-message age, throughput, retry rate, and worker failures.
- Scale consumers based on measured backlog and database capacity, not queue depth alone.

Risk	Design response	Evidence needed
Duplicate delivery	Idempotency key or state transition guard	Repeated message produces no duplicate financial effect.
Worker crashes after commit	Acknowledge only after durable state is confirmed	Re-delivery remains safe.
Poison message	Bounded retry and DLQ	Failure reason and payload reference are visible to operators.
Backpressure	Limit producers or scale workers within database capacity	Queue age and completion rate remain within operating target.
Schema change	Version message contracts and support compatible consumers	Mixed-version rollout is tested.

13. API and Service Integration

APIs should expose stable business contracts rather than database tables. FastAPI is useful for explicit request schemas, validation, generated documentation, and asynchronous integration, but the framework does not define the architecture by itself.

API concern	Design approach	Production implication
Resource ownership	One service owns writes to a business entity or workflow state	Reduces conflicting update paths and unclear responsibility.
Validation	Validate shape, authorization, and domain invariants at the boundary	Rejects invalid work before expensive downstream processing.
Error contracts	Use structured error codes and correlation IDs	Support and clients can distinguish retryable, validation, and internal failures.
Long-running work	Return an investigation or job identifier	Prevents request timeouts and enables progress visibility.
External dependencies	Timeouts, retries, circuit breakers, and response validation	Avoids thread exhaustion and silent acceptance of malformed data.
Versioning	Prefer backward-compatible extension and explicit breaking versions	Supports phased client and service upgrades.

BOUNDARY	The language model, UI, or client should never receive direct database access. Application services validate intent, authorization, and data contracts before executing work.

14. Configuration and Cross-Module Dependencies

The nearly 24-hour eligibility incident showed that configuration is part of the runtime architecture. The Affordable Housing rules and calculations were correct, but a system-level setting introduced or changed by another team altered the result. The dependency was difficult to discover because configuration knowledge was separated from module knowledge.

Configuration capability	Required design property	Why it matters
Scope	Explicit global, company, property, program, or user scope	Prevents ambiguity about where a value applies.
Precedence	Documented and testable override order	Explains which setting wins when multiple values exist.
History	Who changed what, when, and from which release or workflow	Supports incident reconstruction and rollback.
Discoverability	Searchable documentation and dependency mapping	Allows support and engineering to find cross-team effects.
Validation	Prevent incompatible setting combinations	Moves failure detection earlier than customer behavior.
Observation	Include effective configuration in traces or diagnostic views	Reduces manual database investigation.

LESSON	A hidden setting dependency is an architectural defect even when the setting logic itself is correct. Operationally important dependencies must be visible and testable.

15. Document Platform Contracts

Document generation is a contract between template tokens, merge-field definitions, data resolvers, business entities, and generated output. The migration required protecting old and new documents while another team changed the underlying schema.

- Treat merge-field identifiers as public contracts once customer templates depend on them.
- Separate logical field identity from physical database path so schemas can evolve behind an adapter.
- Version templates and resolver behavior when backward compatibility cannot be preserved transparently.
- Validate representative templates across document types, properties, and historical states.
- Capture source entity identifiers, template version, generated object, and generation timestamp for audit.
- Design regeneration as a controlled recovery workflow because corrected data may require replacement documents.

Migration strategy	Advantage	Cost / risk
Big-bang replacement	Simple final state	High blast radius and difficult rollback.
Compatibility adapter	Preserves existing template contracts	Temporary complexity and dual-path maintenance.
Versioned templates	Explicit behavior per version	Requires version governance and migration plan.
Dual-read / comparison	Strong validation before cutover	Additional compute, logging, and temporary implementation.

16. Cloud Object Storage Integration

The AWS S3 migration used a shared migration framework owned by another team. The Affordable Housing responsibility was to discover module document types, adapt mappings, validate metadata and content, support QA, and confirm production behavior. This is a common enterprise architecture model: a platform team owns the shared mechanism while domain teams own correctness of their integration.

Concern	Architecture response	Ownership
Object identity	Deterministic key or durable database reference	Shared platform and domain integration agree on contract.
Metadata	Preserve document type, property, entity, version, and audit references	Domain team validates semantic correctness.
Upload integrity	Checksum, size, content type, and success status	Migration framework provides mechanism; module validates outcomes.
Read compatibility	Application resolves storage location through an abstraction	Prevents callers from depending on physical storage implementation.
Security	Encryption, least-privilege roles, tenant-aware authorization	Platform controls plus application authorization.
Rollback	Preserve source references until verification completes	Release plan determines cutover and recovery.

SCOPE	Owning a domain integration is meaningful architecture work, but it is different from claiming ownership of the shared cloud migration framework.

17. RAG versus Structured Tool Calls

LeaseI intentionally separates unstructured knowledge retrieval from live structured data. Product documentation, regulatory text, release notes, and historical incidents are candidates for retrieval. Current income limits, property settings, feature flags, and customer data should be fetched through trusted structured tools because freshness and exact values matter.

Information type	Preferred access pattern	Reason
Product and engineering prose	Hybrid retrieval over indexed documents	Terminology varies and relevant knowledge may span documents.
HUD/LIHTC regulatory text	Metadata-filtered retrieval with source citations	Requires section, year, jurisdiction, and authority context.
Historical incidents	Semantic + keyword retrieval with confirmed-resolution tier	Symptoms may be paraphrased; resolved causes deserve higher trust.
Current HUD income limits	Live API/tool lookup	Values depend on year, jurisdiction, household size, and program.
Property settings / feature flags	Read-only structured service or database tool	Exact current value and scope are authoritative.
Resident or certification data	Authorized read-only domain API	Sensitive, tenant-scoped, and operationally current.

RULE Do not embed data merely because an LLM is involved. Choose retrieval or tools based on structure, authority, freshness, and access control.

18. LangGraph and Agent Orchestration

LangGraph is appropriate when a workflow is conditional, stateful, multi-step, and must support validation or escalation. A simple question-answering pipeline does not need a graph. LeaseI benefits from a graph because different tickets require different evidence sources and investigation paths.

Node responsibility	Inputs	Outputs / state update
Triage	Ticket description and available context	Category, module, property, required sources, missing fields.
Configuration lookup	Property/module identifiers	Matching settings, scope, values, and release context.
Historical incident search	Symptoms, module, and category	Similar tickets, similarity evidence, and prior resolutions.
HUD lookup	Jurisdiction, household size, year, AMI context	Authoritative limit or structured tool error.
Reasoning	Collected evidence	Ranked hypotheses with evidence references.
Validation	Hypotheses and cited evidence	Supported causes, unsupported claims, and contradictions.
Escalation	Low confidence or incomplete evidence	Structured packet for a human engineer.

In the current build, routing is largely deterministic through graph edges and Python functions. A formal model-selected tool request schema is a target extension rather than an implemented capability.

19. Deterministic versus Probabilistic Responsibility

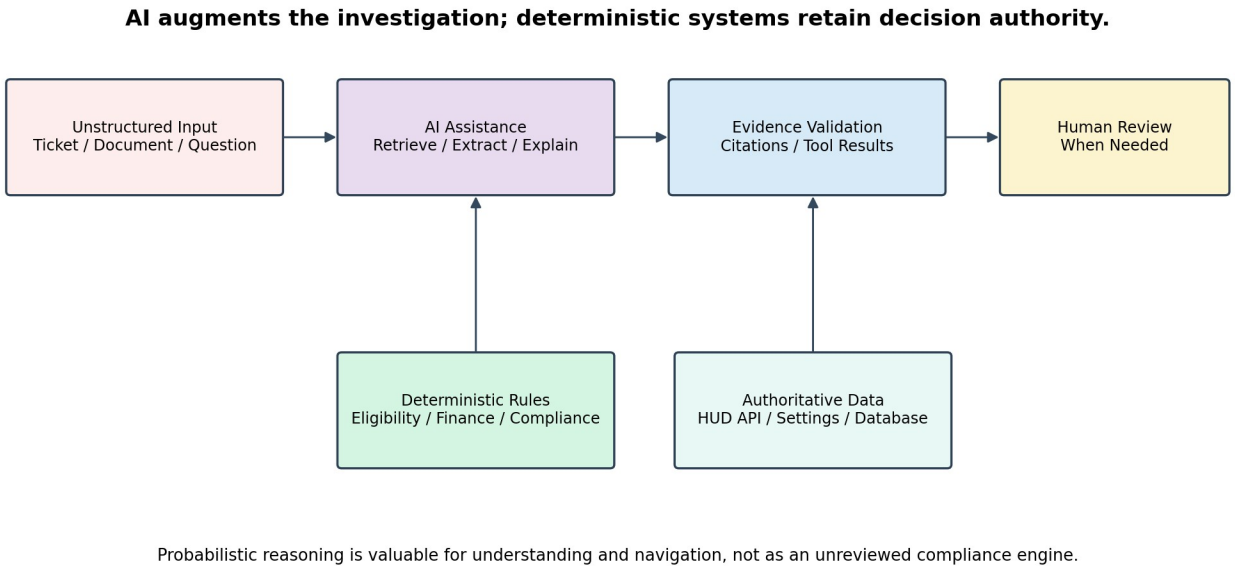


Figure 5. AI supports understanding and investigation; deterministic and human controls retain authority.

Affordable Housing eligibility, financial calculations, and regulated decisions must remain deterministic and auditable. AI can retrieve evidence, extract facts, summarize clauses, correlate incidents, and explain possible causes. It should not silently become the source of truth for a regulated determination.

Responsibility	Best owner	Rationale
Eligibility calculation	Deterministic application engine	Repeatable, testable, and auditable.
Current income limit	Authoritative HUD data tool plus deterministic selection logic	Requires exact dimensional lookup and freshness.
Clause or ticket interpretation	LLM grounded in retrieved sources	Benefits from language understanding.
Configuration value	Read-only structured tool	Requires current exact state.
Root-cause hypothesis ranking	AI reasoning with evidence validation	Useful for correlation but must expose uncertainty.
Final high-impact action	Authorized human or controlled workflow	Maintains accountability and change control.

20. Reliability and Failure Handling

Reliability architecture starts by assuming components will fail: files will be malformed, APIs will time out, workers will crash, messages will be duplicated, migrations will encounter unexpected state, documents will contain legacy mappings, and AI retrieval will sometimes return weak evidence.

Failure type	Detection	Recovery design
Source data malformed	Schema and completeness gate	Reject batch with clear errors; preserve source evidence.
Partial transformation	Batch state, transaction markers, reconciliation mismatch	Rollback, resume from checkpoint, or rerun idempotently.
Duplicate queue delivery	Idempotency key or state transition check	Acknowledge safe duplicate without repeating business effect.
External API unavailable	Timeout, retry metrics, circuit state	Bounded retry, cached policy if allowed, or explicit escalation.
Migration mismatch	Pre/post counts, samples, dumps, timestamp analysis	Stop rollout, compare evidence, roll back or forward-fix.
Document mapping defect	Template regression and production error grouping	Contain affected templates, correct mapping, regenerate safely.
Weak AI evidence	Validation failure or confidence below threshold	Remove unsupported claim and create human escalation.

PRINCIPLE

Retry is not a recovery strategy by itself. Recovery also requires idempotency, bounded attempts, observable state, and an operator path for permanent failure.

21. Performance and Scalability

Performance engineering should begin with a measured bottleneck. The appropriate scaling unit differs by workload: SQL transformations depend on query plans and data shape; RabbitMQ processing depends on consumer and database capacity; APIs depend on latency and concurrency; AI depends on retrieval, model, and tool-call cost.

Workload	Primary scaling lever	Constraint to protect
Income-limit transformation	Set-based SQL, indexes, staging design, batching	Database I/O, locks, temp space, and validation window.
Gross-rent updates	Consumer concurrency and message partitioning strategy	Database contention and duplicate-safe processing.
Document generation	Worker capacity, template caching, object-store throughput	Source-data consistency and regeneration control.
Read APIs	Stateless instances, caching, optimized queries	Freshness, authorization, and dependency latency.
LeaseI investigation	Parallel independent retrieval/tool nodes where safe	Token cost, tool rate limits, evidence consistency.
AgentProbe evaluation	Parallel benchmark execution and result persistence	Model capacity, reproducibility, and cost.

- Measure duration, throughput, queue age, query plans, error rate, and resource utilization before changing architecture.
- Prefer vertical optimization of obviously inefficient queries before adding distributed complexity.
- Use asynchronous scaling only when the downstream database and external systems can sustain the additional concurrency.
- Cache only when staleness semantics are explicit and authorization remains correct.
- Define capacity targets from business windows and expected growth, not arbitrary benchmark numbers.

22. Security, Privacy, and Auditability

The platform processes property, resident, financial, and regulatory information. AI assistance adds the risk of over-broad retrieval and unsafe tool execution. Security must therefore be enforced at every boundary rather than delegated to the user interface or model prompt.

Control	Application to architecture	Evidence
Least privilege	Services and tools receive only required read/write permissions	Role policies and access reviews.
Tenant isolation	Every query and retrieval path carries tenant/property scope	Authorization tests and audit logs.
Read-only diagnostics	LeaseI tools cannot modify production settings or decisions	Tool allowlist and safety tests.
Encryption	Protect data in transit and at rest, including object storage	Configuration and compliance evidence.
Audit trail	Record migrations, configuration changes, investigations, and generated artifacts	Immutable or controlled audit records.
PII minimization	Redact or avoid unnecessary resident details in prompts and logs	Logging policy and data-flow review.
Prompt-injection defense	Treat retrieved documents as untrusted content and restrict tools	Input separation, policy checks, and adversarial tests.

23. Observability and Production Evidence

Observability is what converts architecture assumptions into evidence. The Version 2 migration investigation demonstrated the value of database dumps and timestamp interpretation. The eligibility incident demonstrated the need to expose configuration dependencies. AI systems require traces across retrieval, tool calls, reasoning, and validation.

System area	Signals	Decision enabled
ETL / imports	Batch status, rejects, source/staging/published counts, duration	Publish, retry, rollback, or investigate mapping.
Database	Query duration, plan changes, locks, deadlocks, resource usage	Optimize query, index, transaction, or workload schedule.
RabbitMQ	Queue depth, oldest age, throughput, retries, DLQ, worker failures	Scale, throttle, replay, or fix poison messages.
Documents	Generation rate, failure class, template version, regeneration count	Contain template/mapping defects and validate recovery.
Releases	Migration checkpoints, error rates, business reconciliation, timestamped evidence	Continue rollout, pause, or rollback.
LeaseI	Node latency, retrieval score, tool success, evidence coverage, escalation rate	Improve retrieval, tools, confidence, or workflow.
AgentProbe	Benchmark pass rate, trajectory failures, RAG-stage failures, regression delta	Block deployment or target the failing component.

EVIDENCE	Logs should answer what happened, to which entity, under which configuration and version, and what the system did next. Volume without diagnostic context is not observability.

24. Deployment, Migration, and Rollback

Deployment architecture must include the order of application, database, data, configuration, and document changes. Rollback is not always a single command; a forward data migration may require backups, compatibility periods, or compensating scripts.

1. Classify the release by blast radius, reversibility, data mutation, and cross-team dependency.
2. Capture pre-deployment counts, versions, configuration, and database state needed for comparison.
3. Deploy backward-compatible code before destructive schema changes where possible.
4. Apply migrations in checkpointed phases and validate after each phase.
5. Use canary properties, modules, or controlled data subsets when the business workflow allows it.
6. Run technical checks and business scenario validation before broad rollout.
7. Define explicit pause, rollback, and forward-fix thresholds.
8. Preserve incident evidence before modifying the state during an unexpected result.
9. Complete post-deployment reconciliation and document the final decision.

Change type	Preferred strategy	Rollback concern
Additive schema	Deploy schema, then compatible code	Usually reversible if old readers ignore new fields.
Data backfill	Batch with checkpoints and reconciliation	Must identify and reverse or restore changed rows.
Contract change	Version or compatibility adapter	Mixed-version clients and templates may remain active.
Queue message change	Version message and consumer compatibility	Old messages may remain in flight.
Object-store migration	Dual reference or source preservation until verified	Deleting legacy objects too early removes rollback path.
AI prompt/model/retrieval change	Evaluation gate and controlled rollout	Behavioral regression may not appear as a conventional error.

25. Architecture Evolution and Modernization

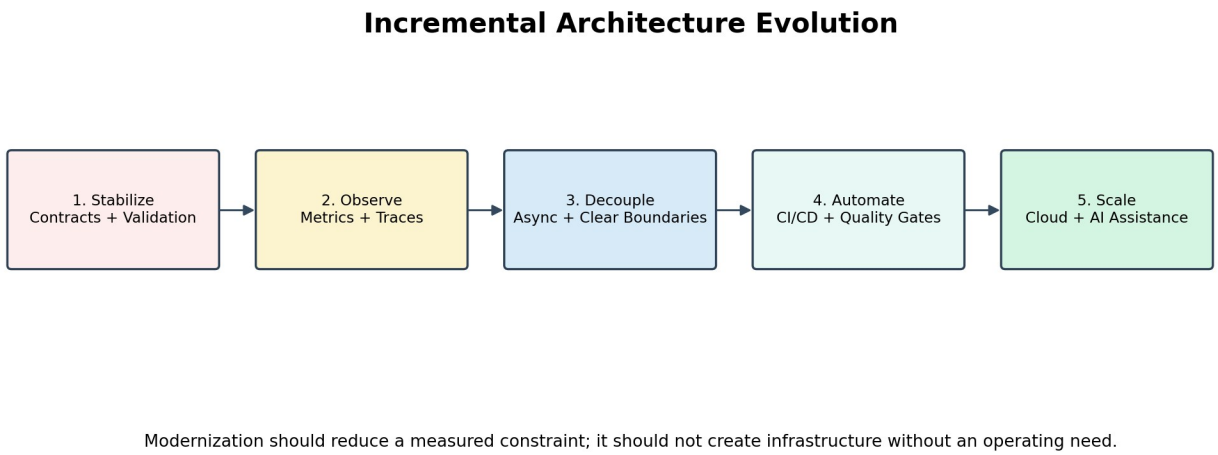


Figure 6. Modernization should be incremental, evidence-based, and tied to an operational constraint.

Modernization is strongest when it starts by stabilizing contracts and visibility. Introducing microservices, queues, cloud infrastructure, or AI before understanding current behavior can increase operational risk. The phased direction below preserves proven domain logic while creating clearer boundaries.

Phase	Changes	Exit evidence
Stabilize	Document contracts, effective dates, settings, validation, and ownership	Known workflows and failures are reproducible.
Observe	Add correlation IDs, batch metrics, queue metrics, configuration history, and reconciliation dashboards	Bottlenecks and failures are measurable.
Decouple	Move long-running work to workers; introduce adapters around documents, storage, and live data	Independent scaling and failure isolation solve a measured problem.
Automate	CI/CD gates for migrations, data checks, documents, and AI benchmarks	Regression is detected before broad release.
Scale	PostgreSQL/pgvector, object storage, Redis or RabbitMQ, stateless APIs, and secure AI tools as justified	Capacity, reliability, and operating ownership are proven.

A modernization component should be introduced only when it removes a demonstrated constraint and has an operating owner; infrastructure should never be added merely to make the design appear modern.

26. Decision Records and Reusable Evaluation Framework

Architecture Decision Records make assumptions and trade-offs explicit. They are especially valuable when a decision crosses teams, affects data, or will be revisited after scale changes.

ADR field	Required content
Context	Business workflow, current pain, scale, dependencies, and constraints.
Decision	Specific boundary, technology, contract, or operating behavior selected.
Options considered	Viable alternatives, including retaining the current design.
Trade-offs	Correctness, complexity, cost, performance, security, and team ownership.
Failure model	Expected failures, detection, retry, rollback, and operator action.
Evidence and metrics	What will prove the decision is working.
Scope boundary	What was not decided and which team owns adjacent concerns.
Review trigger	Volume, latency, incident, cost, or organizational change that requires reevaluation.

Reusable decision scorecard

Dimension	Evaluation question	Warning signal
Correctness	Can the result be reproduced and audited?	Hidden state, duplicated rules, or model-only judgment.
Complexity	Does the option add fewer failure modes than the problem warrants?	Multiple services without independent scale or ownership.
Reliability	Can failures be detected and recovered safely?	Unbounded retry, no idempotency, or no rollback.
Performance	Does the design meet the measured workload window?	Premature scaling or optimization without evidence.
Security	Are authorization and data boundaries enforced at every path?	UI-only checks or broad model/tool access.
Operability	Can the team monitor, deploy, and support it?	Unknown ownership, weak logs, or manual-only recovery.

27. Common Architecture Anti-Patterns

Anti-pattern	Why it fails	Corrective direction
Technology-first design	Solves an imagined architecture problem instead of the business constraint	Write the problem, risks, and success metrics first.
Everything as a microservice	Adds network, deployment, and ownership failure modes without real independence	Use capability and data boundaries; keep cohesive logic together.
Everything as RAG	Treats exact, changing, or sensitive values as approximate text retrieval	Use structured tools for authoritative current data.
LLM as final decision-maker	Probabilistic output becomes unreviewed business truth	Keep deterministic rules and human accountability.
Retry without idempotency	Duplicates business effects and hides permanent failures	Add durable state, bounded retry, and operator handling.
Big-bang migration	Maximizes blast radius and makes evidence difficult to isolate	Use compatibility, phases, checkpoints, and reconciliation.
Database as shared integration API	Couples callers to physical schemas and change timing	Expose service or adapter contracts around owned data.
Configuration without history	Makes production behavior impossible to reconstruct	Store scope, precedence, change history, and release context.
Metrics without context	Produces dashboards that cannot explain a customer issue	Add entity, version, configuration, and correlation identifiers.
Future architecture presented as built	Damages credibility and prevents useful trade-off discussion	Label implemented, partial, and target capabilities clearly.

28. Lessons Learned and Technical Decision Checklist

Lessons learned

- Deep domain understanding is an architectural capability because it reveals hidden data and configuration dependencies.
- Long-term ownership improves design quality by connecting implementation choices to years of production evidence.
- Data transformations require lineage and reconciliation, not only successful execution.
- Asynchronous processing improves responsiveness and failure isolation only when idempotency and observability are designed with it.
- Migrations are temporary distributed systems: old and new versions, schemas, templates, and messages may coexist.
- Configuration is executable behavior and must be versioned, discoverable, and testable.
- AI architecture is still software architecture. Retrieval, tools, authorization, evaluation, and escalation determine trust more than a prompt alone.
- Honest scope boundaries strengthen technical credibility. Domain integration ownership, shared-framework ownership, and target design should never be blurred.
- Production incidents should change architecture through better validation, observability, tests, documentation, or boundaries.
- Simplicity is not the absence of design. It is the result of removing components that do not justify their operational cost.

Technical decision checklist

Before approval	Required answer
Problem	What customer, business, or operational outcome must change?
Authority	Which system or source is the truth for each fact and decision?
Data	What is the model, temporal behavior, lineage, volume, and retention?
Boundary	Which capability owns the state and which contracts are exposed?
Communication	Should the workflow be synchronous, asynchronous, or batch?
Failure	How are partial completion, duplicates, timeouts, and permanent errors handled?
Security	How are authorization, tenant isolation, PII, and audit enforced?
Evidence	What metrics, traces, reconciliation, and tests prove the design works?

Before approval	Required answer
Deployment	How will mixed versions, migration, rollback, and recovery work?
Ownership	Which team can operate, support, and evolve the system?
Scope honesty	Which parts are implemented, generalized, or proposed?
Review trigger	What production evidence will cause the decision to be revisited?

FINAL PRINCIPLE	Architecture is the discipline of making business behavior, data authority, failure handling, and ownership explicit. Technology is selected only after those responsibilities are understood.

End of Case Study